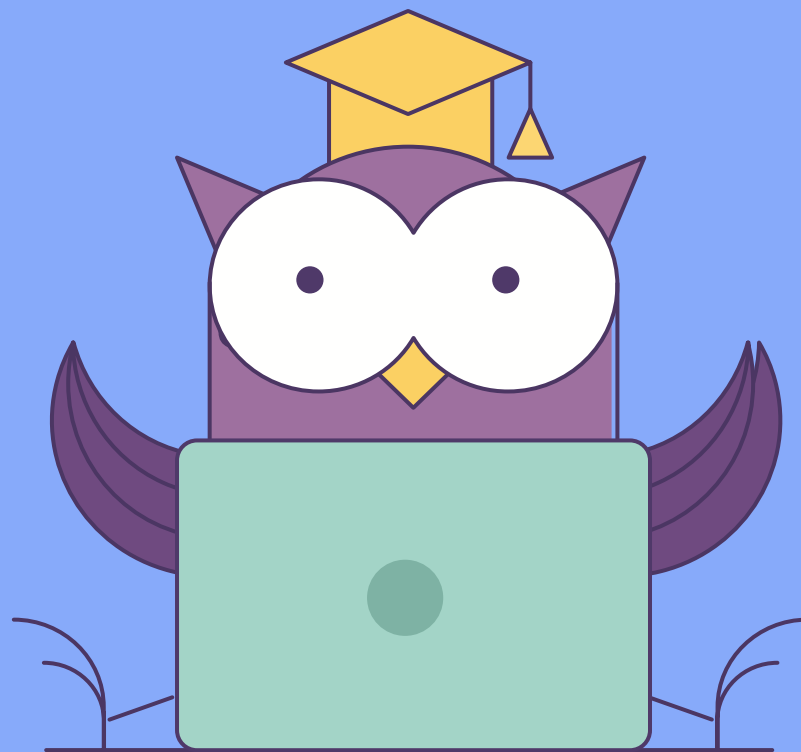




ОНЛАЙН-ОБРАЗОВАНИЕ

Меня хорошо слышно && видно?



Напишите в чат, если есть проблемы!

Ставьте ☐ + если все хорошо
Ставьте ☐ - если есть проблемы

Введение в Spring Framework



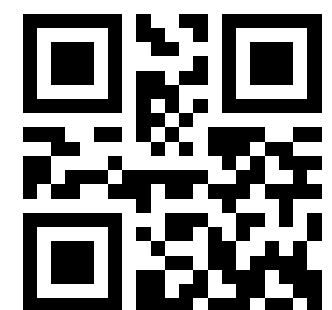
- Разобраться с организационными вопросами.
- ориентироваться в проектах Spring для дальнейшего изучения,
- применять принцип IoC при написании классов и тестов,
- создавать контекст Spring,
- определять Spring Beans в контексте,
- организовывать правильный DI

**Договорились?
Поехали!**

00

Организационные вопросы

- ❖ Как пойдет занятие зависит от того, как будут задаваться вопросы
- ❖ Поэтому желательно вопросы задавать конце блока. Для этого у нас есть специальные слайды
- ❖ Но! Если что-то мешает дальнейшему пониманию темы лучше сразу задать вопрос в чат
- ❖ Оффтоп и выходящие за рамки занятия вопросы лучше задать в группе телеграмма . Или в чат зума. Я их сохраню и отвечу после занятия
- ❖ В конце с вас отзыв)



Александр Оруджев

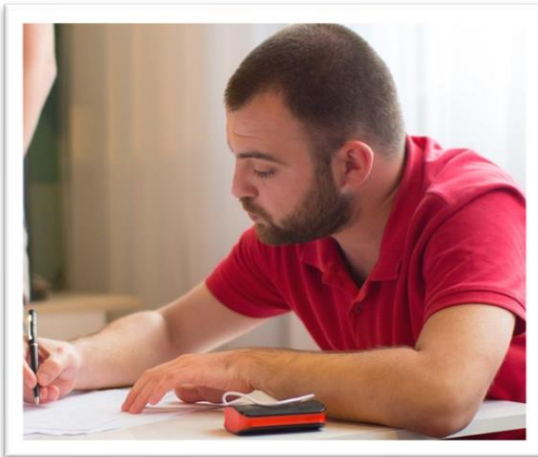
Lead Software Engineer

В отрасли с 2006 года.

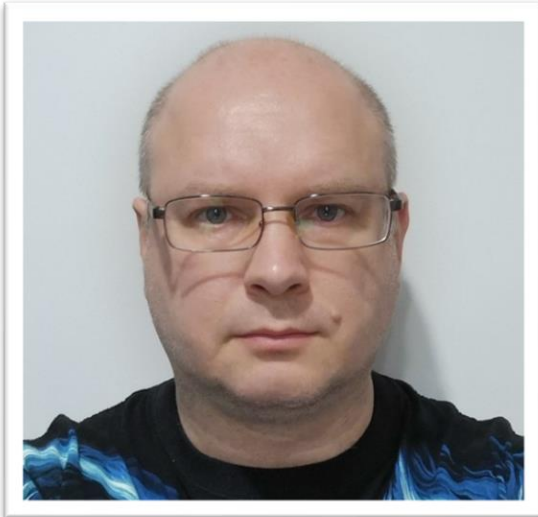
В числе прочего, занимался разработкой системы для информационного сопровождения бурения нефтяных и газовых скважин.

В Отус с 2018 года, работаю на двух курсах

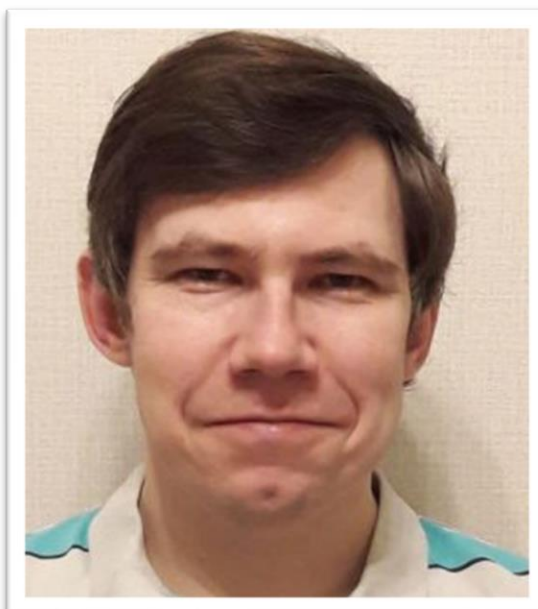




Дворжецкий Юрий – автор программы. Она изменилась с тех пор, но основа та же



Оруджев Александр – текущий руководитель курса, преподаватель, наставник. Ведущий разработчик, Team Lead. 17 лет опыта разработки



Сергей Петрелевич – руководитель курса Java Professional, преподаватель, наставник. Ведущий разработчик. 20 лет опыта разработки



Виталий Куценко – преподаватель, наставник. Java Lead в RND подразделении. 20+ лет опыта разработки



Владимир Иванов – преподаватель, наставник. 20+ лет опыта разработки



Дмитрий Коган – руководитель курса “Углубленное изучение языка Java”, преподаватель. 20+ лет опыта

Демо (ЛК)

Файл: Организационные вопросы.pdf

Что там есть:

- Как все устроено на курсе
- Кому, куда задавать вопросы и какие
- Как происходит сдача ДЗ на проверку



Файл: Правила оформления кода

Что там есть:

- То, к чему мы будем особо придиратся
- Ссылки на соглашения 😊



Файл: Настройка окружения.pdf

Что там есть:

- Что мы работаем на Java 11-17
- Как настроить Git, IntelliJ IDEA и прочее
- Как мы делаем pull-request-ы для ДЗ 😊



Вопросы?

- ставим “+” если вопросы есть
- и “-” если вопросов нет
- далее, добавляем уровень понимания
- это число от 0 до10
- Пример: +7 (почти все понял, есть вопросы)

01

Spring Framework

- Spring – индустриальный стандарт.
- Не побоюсь сказать, что 80% вакансий – так или иначе используют Spring
- Spring – «**lightweight**» фреймворк (объём кода, который необходимо написать программисту – минимален).
- Мы, как раз, и будем учиться эффективно его использовать

- Spring – это фреймворк уровня приложений, на нём можно построить целиком всё приложение.
- Есть «подфреймворки» Spring – проекты Spring, которые отвечают только за один слой приложения.
- Наш курс и построен на изучении проектов Spring

- Spring IoC + AOP = Context
- Spring JDBC
- Spring Data *
- Spring MVC, Spring WebFlux
- Spring Security
- Spring Integration
- Spring Cloud *
- Spring Boot
- И куча всего другого интересного 😊



Spring Framework 5



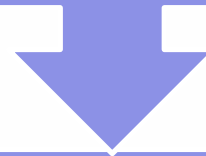
<http://spring.io>

- Документация
- Текущие версии библиотек
- Примеры и tutorials

Вопросы?

- ставим “+” если вопросы есть
- и “-” если вопросов нет
- далее, добавляем уровень понимания
- это число от 0 до10
- Пример: +7 (почти все понял, есть вопросы)

Организационные вопросы



Spring Framework



Spring IoC



Практика



02

Spring IoC

- ❖ Dependency Inversion Principle (DIP) – принцип ООП, согласно которому:
 - модули верхних уровней не должны зависеть от модулей нижних уровней
 - все модули должны зависеть от абстракций
 - абстракции не должны зависеть от деталей
- ❖ Это очень крутой принцип, буква “D” из SOLID
- ❖ Но, напрямую, не относится к понятиям “Inversion of Control” и “Dependency Injection”
- ❖ Хотя и имеет в названии, с каждым из них, по одному общему слову

- ❖ Inversion of Control (IoC) – принцип ООП, согласно которому поток управления программы контролируется фреймворком или просто чужим кодом
- ❖ Еще одно название «Голливудский принцип» (Hollywood Principle): «Не звоните мне, я сам Вам позвоню.»
- ❖ IoC может принимать разные формы
- ❖ В частности паттерн «TemplateMethod», лямбды и коллбеки тоже являются частными случаями IoC

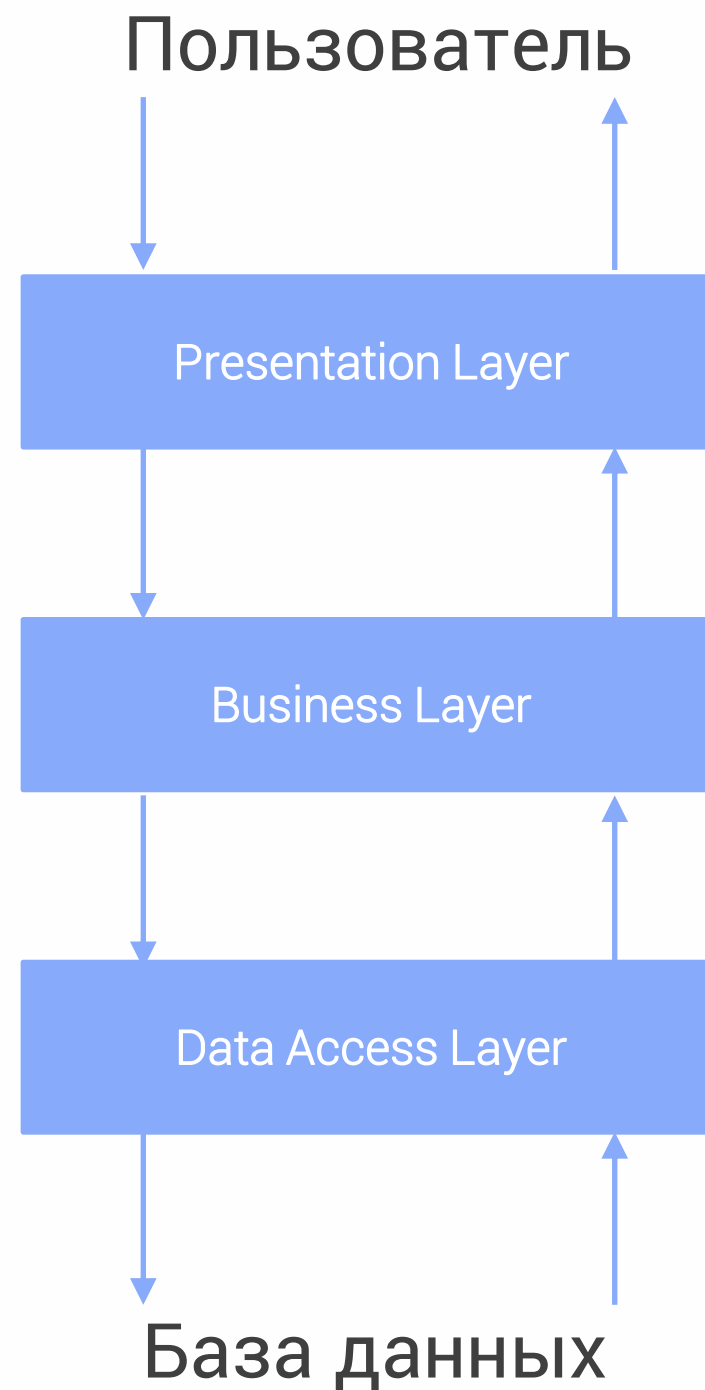
```
public class AnyTest {  
    public static void globalSetUp() {...}  
  
    public void setUp() {}  
  
    public void anyTest1() {...}  
  
    public void anyTest2() {...}  
  
    public void tearDown() {...}  
  
    public static void globalTearDown() {...}  
}
```

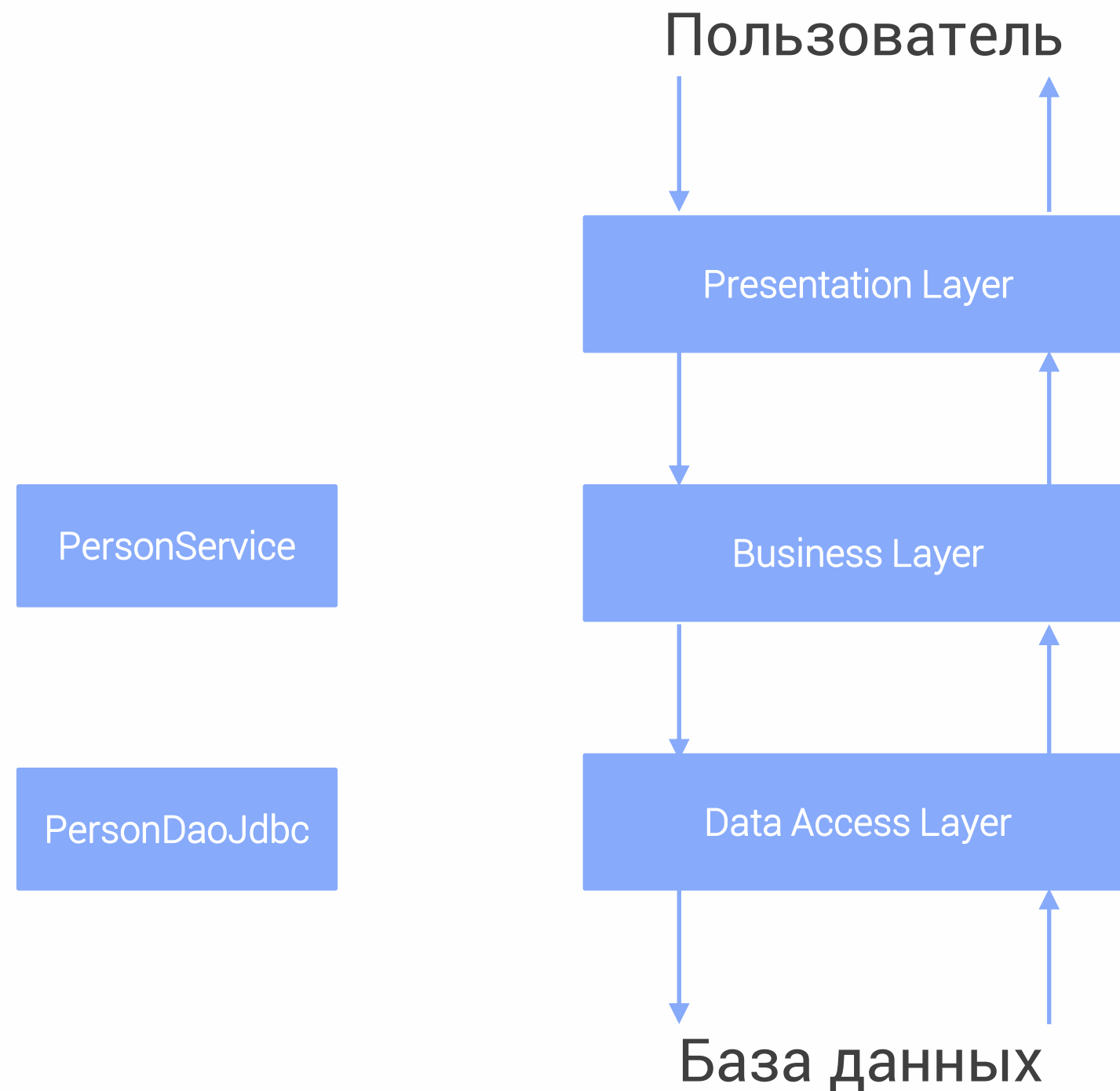
```
public class AnyTestRunner {  
    public static void main (String[] args) {  
        AnyTest test = new AnyTest();  
  
        test.globalSetUp();  
  
        test.setUp();  
        test.anyTest1();  
        test.tearDown();  
  
        test.setUp();  
        test.anyTest2();  
        test.tearDown();  
  
        test.globalTearDown();  
    }  
}
```

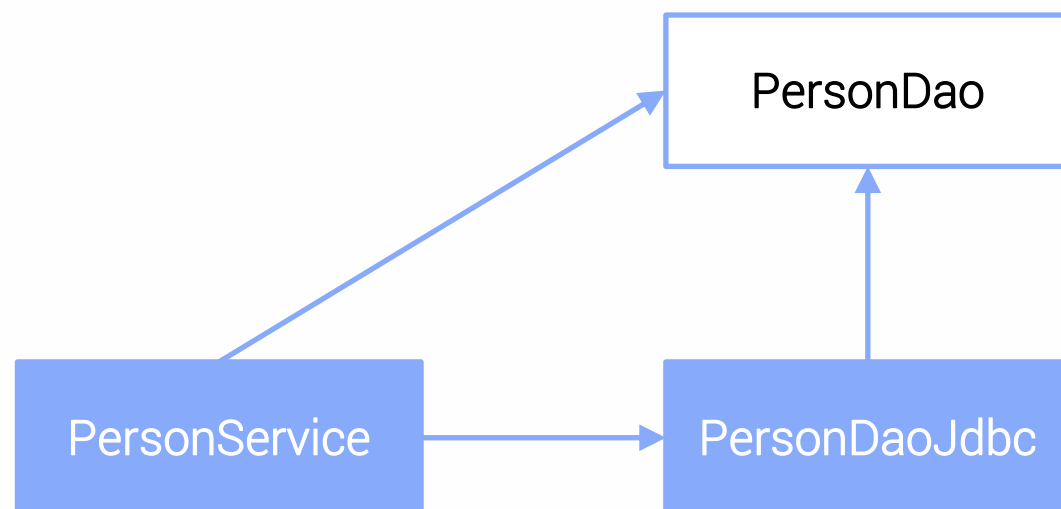
```
public class AnyTest {  
    @BeforeAll  
    public static void globalSetUp() {...}  
  
    @BeforeEach  
    public void setUp() {}  
  
    @Test  
    public void anyTest1() {...}  
  
    @Test  
    public void anyTest2() {...}  
  
    @AfterEach  
    public void tearDown() {...}  
  
    @AfterAll  
    public static void globalTearDown() {...}  
}
```

- ❖ Dependency Injection (DI) – одна из реализаций принципа IoC
- ❖ По сути это инвертирование потока сборки приложения из компонентов
- ❖ IoC-контейнер – это как раз часть DI фреймворка, которая такой сборкой занимается
- ❖ И как следствие хранит в себе все компоненты приложения и обеспечивает их жизненный цикл

- ❖ Говорим IoC/DI-контейнер – подразумеваем контекст и наоборот
- ❖ В контексте хранятся бины, он же их вяжет друг с другом (DI)
- ❖ Бины, тоже самое, что компоненты
- ❖ Компоненты это то из чего состоит приложение, то чем оно манипулирует, компонентом не является
- ❖ Аналогия с домом: подъезды, этажи, квартиры и комнаты, являются частью (компонентом) дома, а люди, питомцы, мебель и вещи – нет
- ❖ Вывод, людей с вещами и питомцами нельзя класть в контекст 😊







```
class PersonService {

    private final PersonDao dao;

    public PersonService() {
        this.dao = new PersonDaoJdbc(
            "127.0.0.1:80"
        );
    }

    public Person getByName(...) { ... }
}

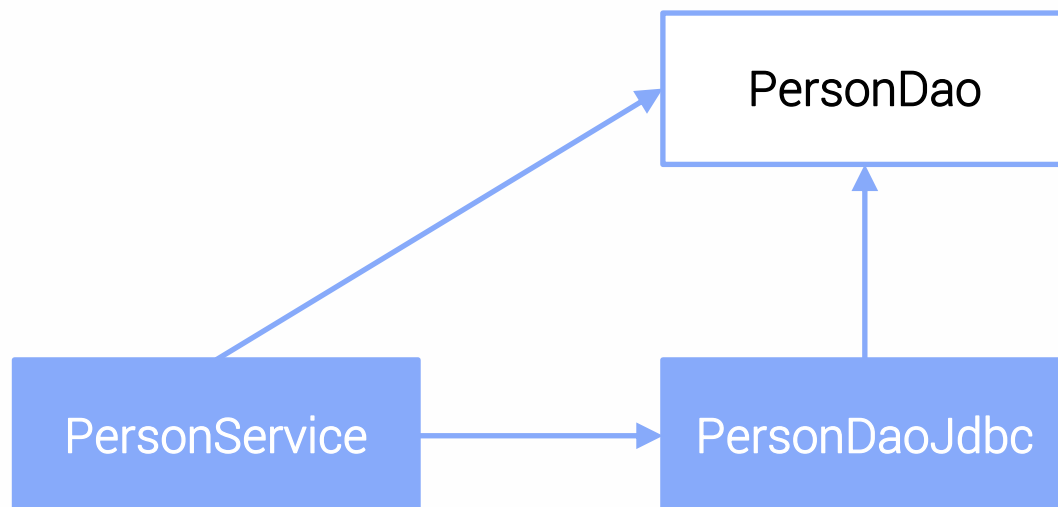
class PersonDaoJdbc implements PersonDao {

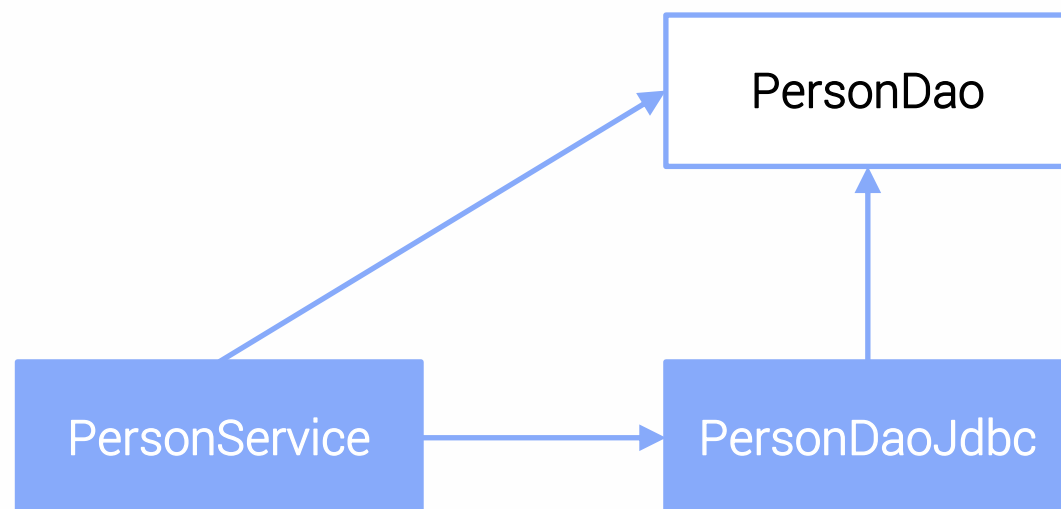
    private final String url;

    public PersonDaoJdbc(String url) {
        this.url = url;
    }

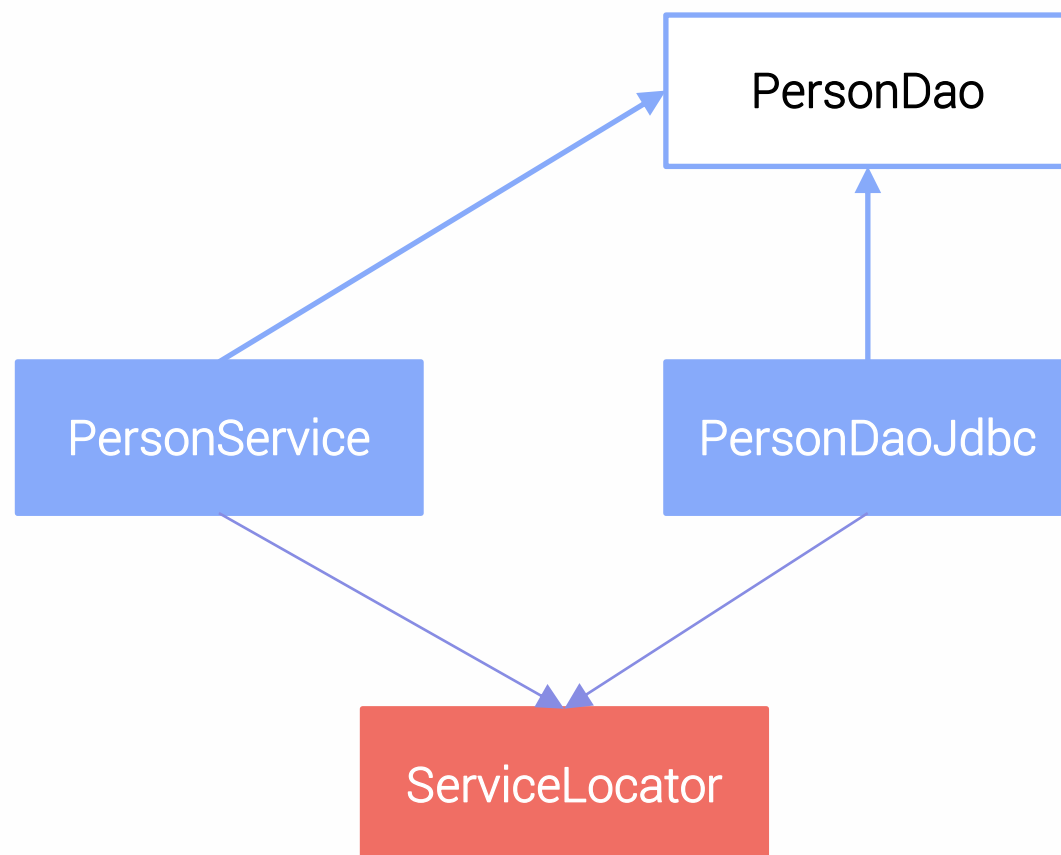
    public Person findByName(...) { ... }
}
```


- Чем хорош?
- Чем плох?





- Класс PersonService напрямую зависит от PersonDaoJdbc.
- Невозможно тестировать PersonService отдельно от PersonDaoJdbc.
- Жизненный цикл классов связан напрямую.
- Невозможно заменить PersonDaoJdbc на другую реализацию.



```
class PersonService {

    private final PersonDao dao;

    public PersonService() {
        this.dao = ServiceLocator
            .getPersonDao();
    }

    public Person getByName(...) { ... }
}

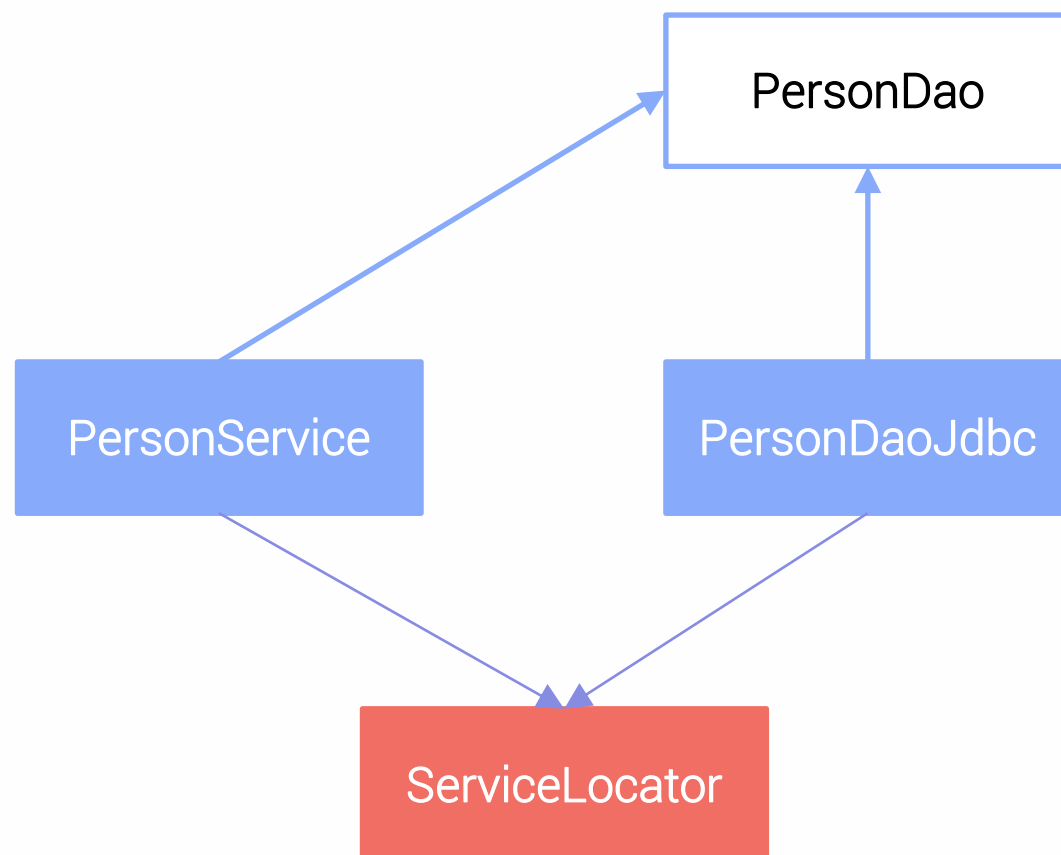
class PersonDaoJdbc implements PersonDao {

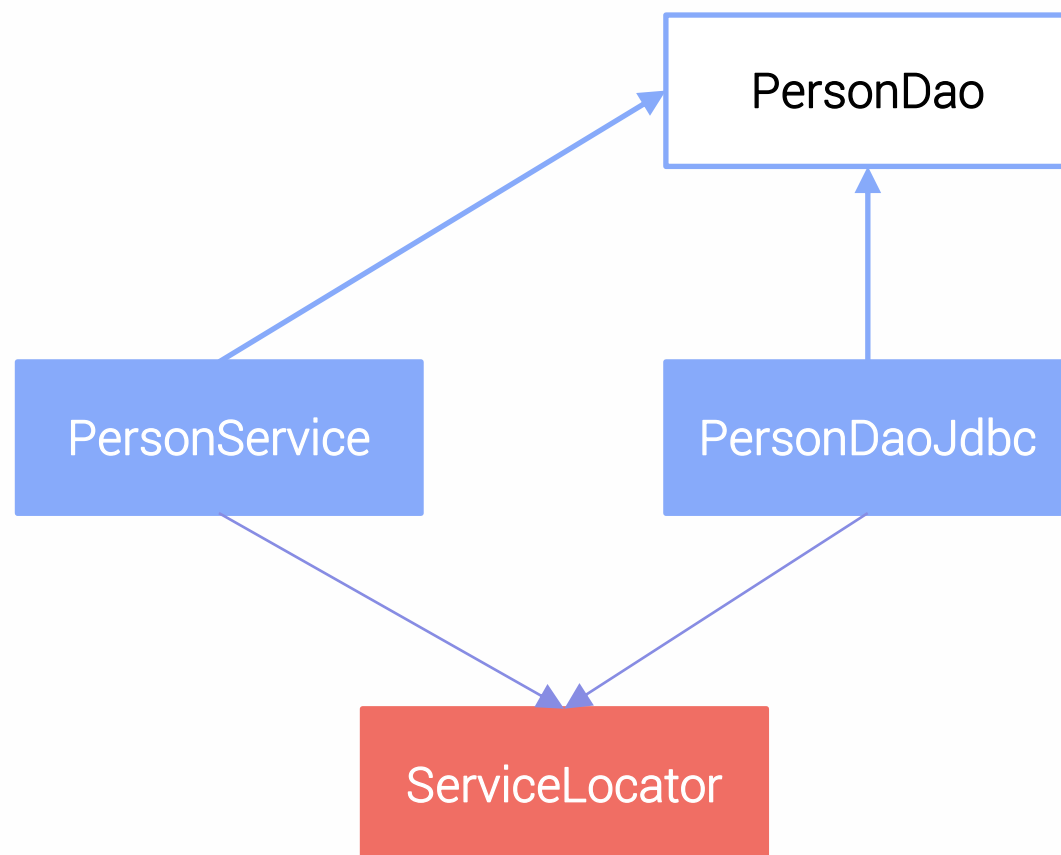
    private final String url;

    public PersonDaoJdbc(String url) {
        this.url = url;
    }

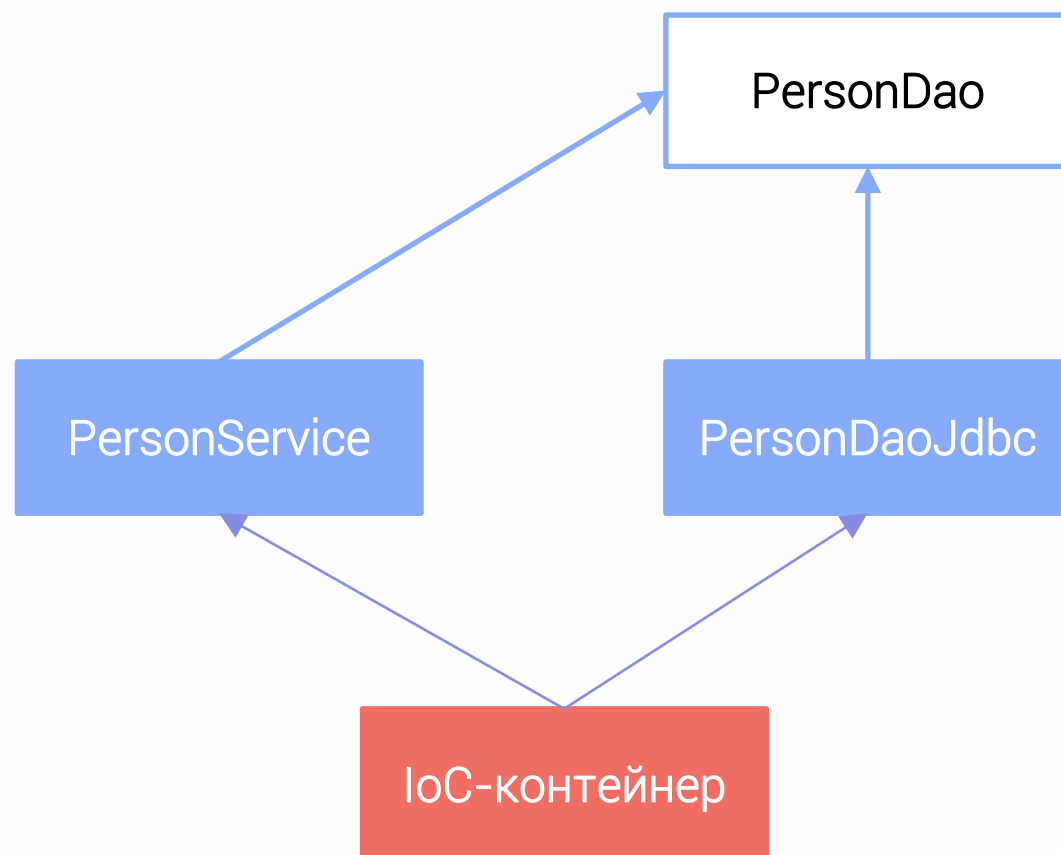
    public Person findByName(...) { ... }
}
```

- Чем хорош?
- Чем плох?



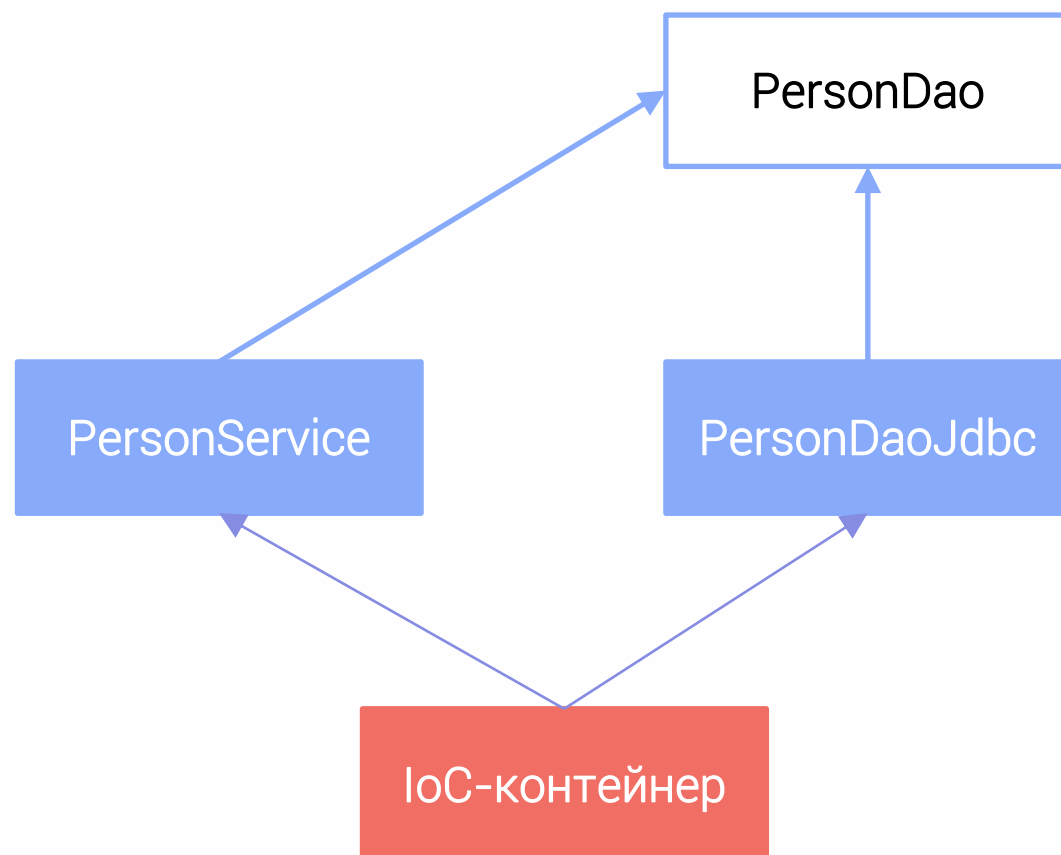


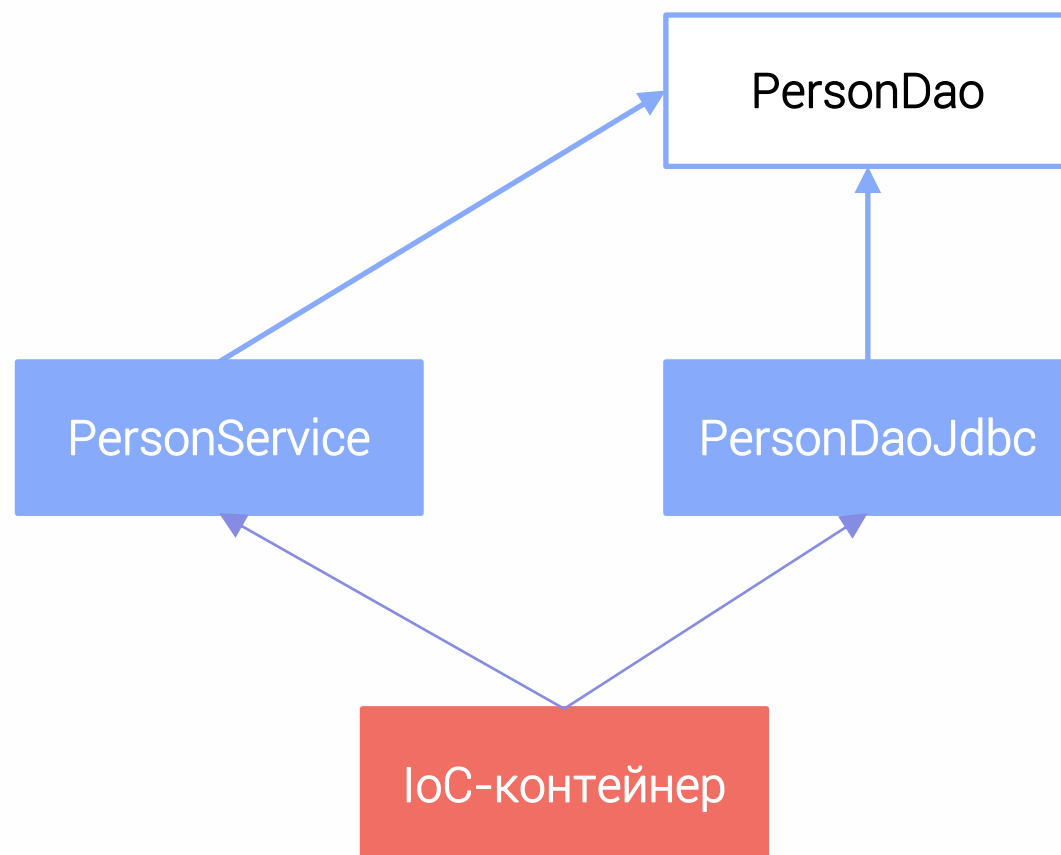
- Класс **PersonService** не зависит от **PersonDaoJdbc**, но зависит от **ServiceLocator**-а.
- Если постараться, то **PersonService** можно тестировать отдельно от **PersonDaoJdbc**.
- **PersonDaoJdbc** можно заменить на другую реализацию.



```
class PersonService {  
  
    private final PersonDao dao;  
  
    public PersonService(PersonDao d) {  
        this.dao = d;  
    }  
  
    public Person getByName(...) { ... }  
}  
  
class PersonDaoJdbc implements PersonDao {  
  
    private final String url;  
  
    public PersonDaoJdbc(String url) {  
        this.url = url;  
    }  
  
    public Person findByName(...) { ... }  
}
```

- Чем хорош?
- Чем плох?





- Контейнер создаёт необходимые объекты и управляет жизненным циклом
- Класс PersonService не зависит от PersonDaoJdbc
- PersonService можно легко тестировать отдельно от PersonJdbc
- PersonDaoJdbc можно заменить на другую реализацию


```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns="http://www.springframework.org/schema/beans"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="personDao" class="ru.otus.spring.dao.PersonDaoJdbc">
        <constructor-arg name="dbUrl" value="${db.url}"/>
    </bean>

    <bean id="personService" class="ru.otus.spring.service.PersonService">
        <constructor-arg name="dao" ref="personDao"/>
    </bean>

</beans>
```

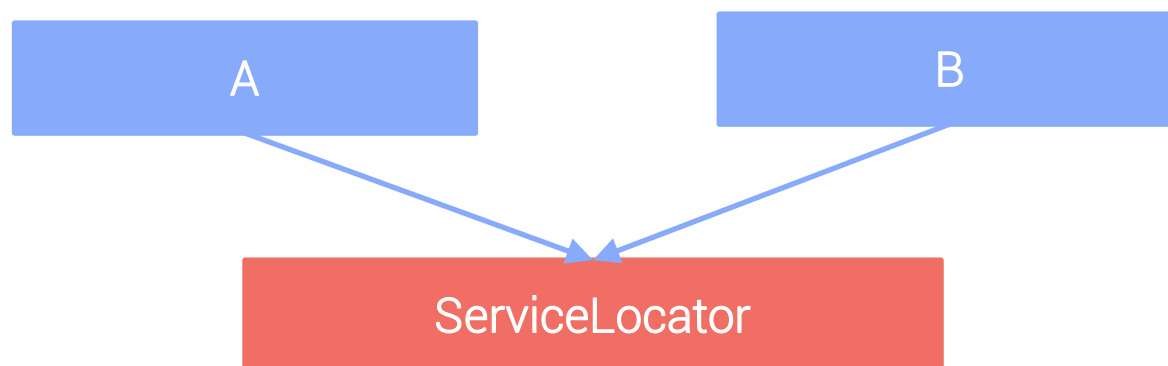
```
@Configuration
class AppConfig {

    @Bean
    PersonDao personDao(@Value("${db.url}") String dbUrl) {
        return new PersonDaoJdbc(dbUrl);
    }

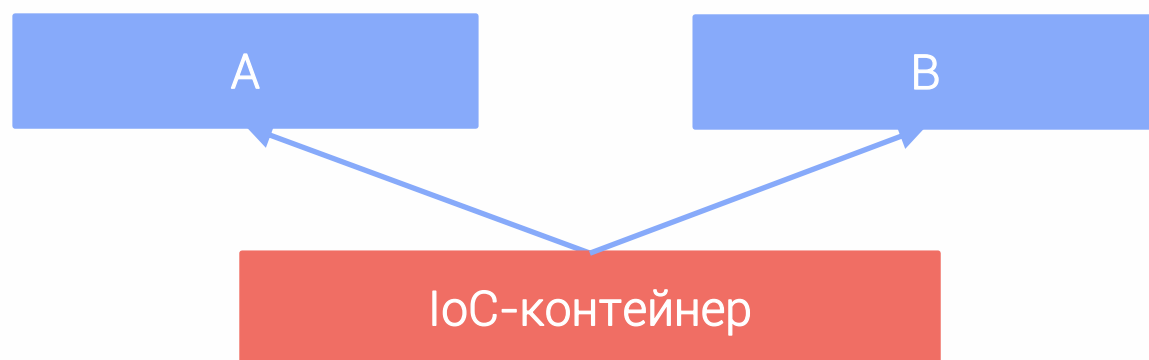
    @Bean
    PersonService personService(PersonDao dao) {
        return new PersonService(dao);
    }
}
```



Классический подход



Service Locator



IoC-контейнер

```
class PersonService {  
  
    private final PersonDao dao;  
  
    public PersonService() {  
        this.dao = ServiceLocator  
            .getPersonDao();  
    }  
  
    public Person getByName(...) { ... }  
}  
  
class PersonDaoJdbc implements PersonDao {  
  
    private final String url;  
  
    public PersonDaoJdbc(String url) {  
        this.url = url;  
    }  
  
    public Person findByName(...) { ... }  
}
```

```
class PersonService {  
  
    private final PersonDao dao;  
  
    public PersonService(PersonDao d) {  
        this.dao = d;  
    }  
  
    public Person getByName(...) { ... }  
}  
  
class PersonDaoJdbc implements PersonDao {  
  
    private final String url;  
  
    public PersonDaoJdbc(String url) {  
        this.url = url;  
    }  
  
    public Person findByName(...) { ... }  
}
```

- Упрощает reuse компонентов.
- Упрощает unit-тестирование – это у нас важно на курсе 😊
- Чистый код – только бизнес-логика, никакого конфигурационного кода.

Вопросы?

- ставим “+” если вопросы есть
- и “-” если вопросов нет
- далее, добавляем уровень понимания
- это число от 0 до10
- Пример: +7 (почти все понял, есть вопросы)

- Бывают разные.
- Конфигурируются разными способами.
- Конфигурируются разными языками XML, Java, Groovy.

IoC-контейнеры в Spring

- XML является традиционным способом задания конфигурации контейнера, хотя существуют и другие способы задания метаданных (аннотации, Java код и т.д.).
- Во многих случаях проще и быстрее конфигурировать контейнер с помощью аннотаций. (это мы пройдем позже).
- Первое занятие и первое ДЗ мы будем конфигурировать с помощью XML, дальше забудем как страшный сон.

Сначала читаются
метаданные



По метаданным читаются
все необходимые классы



Создаются все бины



Задаются зависимости

```
public class PersonServiceImpl implements PersonService {  
  
    private final PersonDao dao;  
  
    public PersonServiceImpl(PersonDao dao) {  
        this.dao = dao;  
    }  
  
    public Person getByName(String name) {  
        return dao.findByName(name);  
    }  
}
```

```
<dependency>  
  <groupId>org.springframework</groupId>  
  <artifactId>spring-context</artifactId>  
  <version>X.X.X</version>  
</dependency>
```

```
public class Main {  
    public static void main(String[] args) {  
        ClassPathXmlApplicationContext context =  
            new ClassPathXmlApplicationContext("/spring-context.xml");  
        PersonService service = context.getBean(PersonService.class);  
        Person ivan = service.getByName("Ivan");  
    }  
}
```

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xmlns="http://www.springframework.org/schema/beans"
        xsi:schemaLocation="http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="personDao" class="ru.otus.spring.dao.PersonDaoSimple">
    </bean>

    <bean id="personService" class="ru.otus.spring.service.PersonServiceImpl">
        <constructor-arg name="dao" ref="personDao"/>
    </bean>

</beans>
```

Ваши вопросы?

03

Демо

Ссылка: GitHub - репозиторий

<https://github.com/OtusTeam/Spring>

Здесь будут располагаться все упражнения,
которые мы будем делать на занятии



- Посмотреть реализацию Spring IOC на XML
- Выкачиваем репозиторий и просто смотрим

```
1. git clone git@github.com:OtusTeam/Spring.git
   или
   git clone https://github.com/OtusTeam/Spring.git

2. cd ./<имя группы>/spring-01

3. Смотрим
   spring-01-solution/src/main/resources/spring-context.xml

4. Число 1 в чат, как посмотрите
```

Слушатели, которые смотрят нас в записи, могут поставить паузу.

- Добавить Maven-зависимость spring-context в pom.xml

1. Посмотреть на <http://mvnrepository.com> текущую версию org.springframework:spring-context

2. Добавляем в **spring-01-exercise/pom.xml**

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
  <version>??????????</version>
</dependency>
```

3. Число 2 в чат, как сделаете

Слушатели, которые смотрят нас в записи, могут поставить паузу.

- Создать контекст в main-методе
- Получить PersonService из контекста (его ещё там нет)
- Получить Person из PersonService

1. Добавляем в **spring-01-exercise/src/main/java/ru/otus/spring/Main.java**

```
ClassPathXmlApplicationContext context =  
    new ClassPathXmlApplicationContext("/spring-context.xml");  
PersonService service = context.getBean(PersonService.class);  
Person ivan = service.getByName("Ivan");
```

2. Число 3 в чат, как сделаете

Слушатели, которые смотрят нас в записи, могут поставить паузу.

- Добавить бины в контекст.
- Настроить зависимость через <constructor-arg ref=.

1. Добавляем в
spring-01-exercise/src/main/resources/spring-context.xml

```
<bean id="personDao"  
      class="ru.otus.spring.dao.PersonDaoSimple">  
</bean>  
<bean id="personService"  
      class="ru.otus.spring.service.PersonServiceImpl">  
  <constructor-arg name="dao" ref="personDao"/>  
</bean>
```

2. Число 4 в чат, как сделаете.

Вопросы?

- ставим “+” если вопросы есть
- и “-” если вопросов нет
- далее, добавляем уровень понимания
- это число от 0 до10
- Пример: +7 (почти все понял, есть вопросы)

```
public class PersonServiceImpl implements PersonService {  
  
    private PersonDao dao;  
  
    public PersonServiceImpl() {  
    }  
  
    public void setDao(PersonDao dao) {  
        this.dao = dao;  
    }  
  
    public Person getByName(String name) {  
        return dao.findByName(name);  
    }  
}
```

DI через property (setter)

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns="http://www.springframework.org/schema/beans"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="personDao" class="ru.otus.spring.dao.PersonDaoSimple">
    </bean>

    <bean id="personService" class="ru.otus.spring.service.PersonServiceImpl">
        <property name="dao" ref="personDao"/>
    </bean>

</beans>
```


- Поправить классы зависимостей
- Настроить зависимость через `<property ref=`

1. Правим **PersonServiceImpl.java**
2. Правим **spring-context.xml**
3. Число 5 в чат, как сделаете.

Слушатели, которые смотрят нас в записи, могут поставить паузу.

Вопросы?

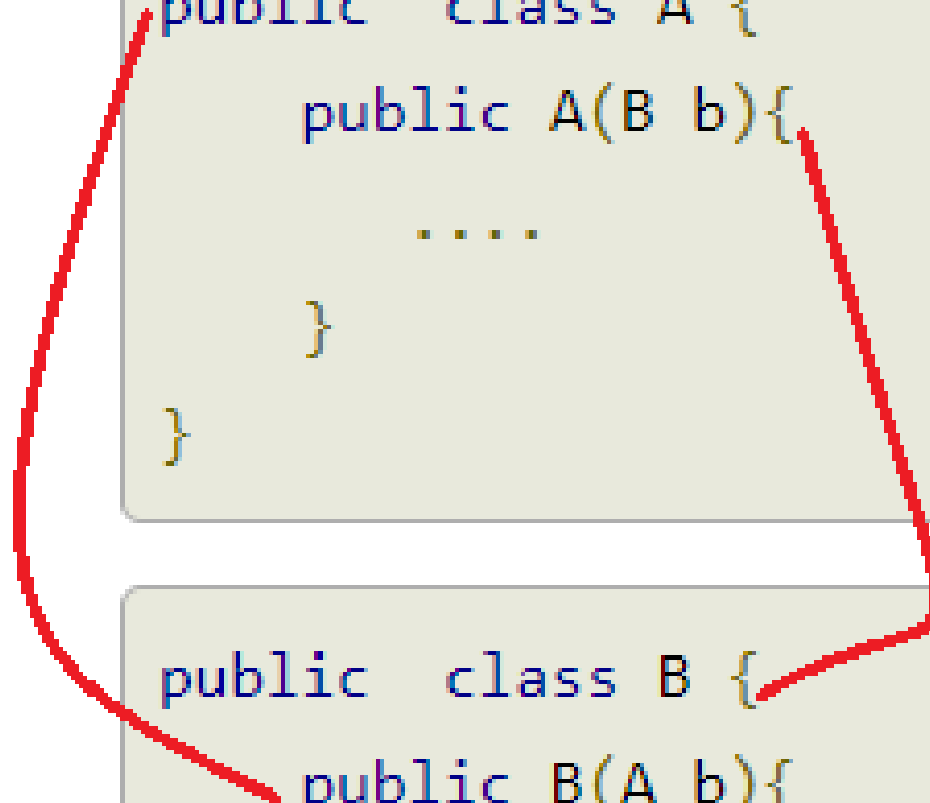
- ставим “+” если вопросы есть
- и “-” если вопросов нет
- далее, добавляем уровень понимания
- это число от 0 до10
- Пример: +7 (почти все понял, есть вопросы)

- DI через конструктор – рекомендован.
- Но могут быть циклические зависимости (виноваты в этом – Вы!).
- Вы всегда получаете готовый к работе класс.
- С property может быть NPE.

Cyclic dependency

```
public class A {  
    public A(B b){  
        ....  
    }  
}
```

```
public class B {  
    public B(A b){  
        ....  
    }  
}
```



```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns="http://www.springframework.org/schema/beans"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="personDao" class="ru.otus.spring.dao.PersonDaoSimple">
        <!-- 18 в PersonDaoSimple заменить на поле, задаваемое здесь -->
        <property name="defaultAge" value="28"/>
    </bean>

    <bean id="personService" class="ru.otus.spring.service.PersonServiceImpl">
        <property name="dao" ref="personDao"/>
    </bean>

</beans>
```

Вопросы?

- ставим “+” если вопросы есть
- и “-” если вопросов нет
- далее, добавляем уровень понимания
- это число от 0 до10
- Пример: +7 (почти все понял, есть вопросы)

- То из чего состоит приложение
- Сервисы, Дао, Репозитории
- Источники подключений к внешним системам и БД (DataSource)
- Мапперы/Маршаллеры/конвертеры;
- Служебные и инфраструктурные бины (PersistenceContextManager...)
- Бизнес-бины стратегий (Паттерн стратегия)

- Бизнес-объекты (бины, пользователи*)
- Поштучные настройки, кроме пачек/файлов настроек*
- Стандартные классы (String, InputStream, Scanner, Locale*)
- В т.ч. в домашнем задании, которое сейчас задам 😊

Вопросы?

- ставим “+” если вопросы есть
- и “-” если вопросов нет
- далее, добавляем уровень понимания
- это число от 0 до10
- Пример: +7 (почти все понял, есть вопросы)

Программа по проведению тестирования студентов

- В ресурсах хранятся вопросы и различные ответы к ним в виде CSV файла.
- Программа должна спросить у пользователя фамилию и имя, спросить вопросы из CSV-файла и вывести результат тестирования.

В рамках этого ДЗ (первая часть):

- Только чтение и вывод вопросов (!)

Описание части на это ДЗ:

- В ресурсах хранятся вопросы и различные ответы к ним в виде CSV файла (5 вопросов).
- Вопросы могут быть с выбором из нескольких вариантов или со свободным ответом - на Ваше желание и усмотрение.
- Приложение должно просто вывести вопросы теста из CSV-файла с возможными вариантами ответа.

Требования:

0. В приложении должна присутствовать объектная модель (отдаём предпочтение объектам и классам, а не строкам и массивам/спискам строчек).
1. Все классы в приложении должны решать строго определённую задачу (см. п. 18-19 "Правила оформления кода.pdf", прикреплённые к материалам занятия).
2. Контекст описывается XML-файлом.
3. Все зависимости должны быть настроены в IoC контейнере.
4. Имя ресурса с вопросами (CSV-файла) необходимо захардкодить строчкой в XML-файле с контекстом.

5. CSV с вопросами читается именно как ресурс, а не как файл.
6. Scanner, PrintStream и другие стандартные типы в контекст класть не нужно!
7. Весь ввод-вывод осуществляется на английском языке.
8. Крайне желательно написать юнит-тест какого-нибудь сервиса (оцениваться будет только попытка написать тест).
9. Помним – «без фанатизма» 😊

Опционально (задание со "звёздочкой"):

1*. Приложение должно корректно запускаться с помощью "java -jar"

Заготовка для выполнения работы:

<https://github.com/OtusTeam/Spring/tree/master/templates/hw01-xml-config>

Описание части на это ДЗ:

- Практически все приложения-ДЗ на курсе можно сделать `stateless`, т.е. без состояния
- По крайней мере на низких и средних уровнях
- А значит так и надо попробовать сделать

Демо (checkstyle)

Факт сдачи:

0 – 1 - задание не сдано / сдано

Степень выполнения (value, количество работающего функционала, что примет заказчик, что будет проверять тестировщик):

0 – 4 - ничего не работает или отсутствует основной функционал /
основной функционал есть, всё хорошо работает, тесты и/или задание перевыполнено

Способ выполнения (code quality, качество выполнения, стиль кода, как ревью перед мержем):

0 – 5 - нужно править, мержить нельзя (нарушение соглашений) /
отличная работа + экстра балл*

Статус "Принято" ставится от 6 и выше баллов.

Идеальное, но не работающее, решение = 5 - не принимается.

Если всё работает, но стилю не соответствует (публичные поля, классы капсом) = 5 - тоже не принимается

Видео: ЛикБез – Unit-тестирование

Что там есть:

- Какие тесты бывают
- Как писать именно Unit-тесты
- Mockito, TDD, BDD – вот это вот всё
- Это полноценное занятие



Видео: ЛикБез – Введение в Maven

Что там есть:

- Введение в Maven)



- Разобрались с вопросами, где писать, куда сдавать ДЗ 😊
- Что такое Spring
- Что такое DI
- Как создавать контекст со Spring
- Ну и как конфигурировать бины в XML
- И получили первое домашнее задание 😊

Вопросы?

- ставим “+” если вопросы есть
- и “-” если вопросов нет
- далее, добавляем уровень понимания
- это число от 0 до10
- Пример: +7 (почти все понял, есть вопросы)

Напишите в группу

- Как Вас зовут;
- Ваш опыт в Java;
- Ваш опыт с Maven || Gradle;
- Ваша степень знакомства со Spring;
- Ваши ожидания от курса.



**Пожалуйста, пройдите
опрос**

<https://otus.ru/polls/83752/>

Спасибо за внимание!
ЮС в помощь!

